

Poker-AD: Meta-Gaming via In-Context Algorithm Distillation in Imperfect-Information Games

Anonymous Author
Affiliation
Address
email

Abstract

Imperfect-information games (IIGs) such as Poker are a canonical testbed for multi-agent decision making under partial observability, strategic deception, and non-stationary opponents. State-of-the-art systems like DeepStack and Pluribus approximate Nash equilibrium strategies that are provably robust but inherently conservative: they do not actively *exploit* suboptimal opponents and therefore leave value on the table. Existing opponent-exploitation methods either maintain explicit Bayesian beliefs over handcrafted opponent types or rely on online reinforcement learning, both of which are sample-inefficient and too slow to adapt within realistic match horizons.

We propose **Poker-AD** (*Poker Algorithm Distillation*), a Transformer-based agent that treats opponent exploitation itself as an *algorithm* to be learned via in-context meta-learning. Offline, we construct a *Hindsight Oracle Teacher* that has full access to an opponent’s policy and computes an approximate best response using CFR-based solvers. We then distill the teacher’s behavior into a causal Transformer that only observes the same partial histories as a human player: public actions, rewards, and occasional showdowns. At test time, Poker-AD performs implicit Bayesian opponent inference and best-response planning entirely inside its forward pass, achieving *zero-shot* adaptation without any gradient updates or decision-time search.

On Leduc and Flop Hold’em Poker, Poker-AD (i) significantly outperforms static GTO baselines and online PPO in head-to-head matches against a diverse opponent zoo, (ii) adapts to new opponent styles in fewer than 10 hands, and (iii) generalizes to held-out mixtures and non-stationary opponents. Ablations show that showdown information, long-range context, and the sequence-modeling architecture are all crucial: removing any of these components dramatically degrades exploitative performance. Representation probing further reveals that opponent parameters are linearly decodable from the Transformer’s hidden states, suggesting that Poker-AD has learned an amortized, in-context approximation to Bayesian opponent modeling in adversarial POMDPs.

1 Introduction

Imperfect-information games (IIGs) such as poker, contract bridge, and security games occupy a central role in artificial intelligence. They capture many phenomena that are

difficult to model in single-agent Markov decision processes: hidden information, partial observability, strategic deception, and the non-stationarity induced by other learning agents. Poker in particular has driven substantial progress in multi-agent reinforcement learning, game theory, and search [3, 6, 10].

Over the last decade, systems such as DeepStack [8] and Pluribus [2] have reached and surpassed top human professionals in various no-limit Texas Hold'em variants. These systems approximately compute Nash equilibrium (NE) strategies via counterfactual regret minimization (CFR) [3? ?], value function approximation, and subgame solving. In two-player zero-sum settings, NE strategies are robust: they guarantee non-negative expected value against any opponent strategy.

However, robustness is not the same as optimal performance against *actual* opponents. NE strategies are by construction indifferent to the opponent's mistakes. When facing suboptimal, human-like or stylized play, a purely game-theoretic optimal (GTO) agent typically remains near break-even rather than actively *exploiting* systematic leaks. By contrast, experienced human professionals treat equilibrium play as a baseline and then dynamically deviate to exploit observed biases within a relatively small number of hands.

The challenge of fast opponent exploitation. Formally, opponent exploitation in IIGs requires simultaneously:

- **Inference under partial observability:** Opponent policies are not directly observable; only public actions and occasional revealed private information (at showdown) are available.
- **Adaptation under tight sample budgets:** In many practical settings, agents must meaningfully adapt within tens to hundreds of hands, far below the sample complexity of standard online RL.
- **Robustness to misspecification:** Attempts to exploit can themselves become exploitable if the opponent model is incorrect or if the opponent adapts.

Classical opponent modeling approaches maintain explicit Bayesian beliefs over a finite set of handcrafted opponent types and compute best responses to the posterior [11?]. While principled, such methods are brittle to misspecified type spaces and do not scale well to large games. Alternatively, online reinforcement learning approaches treat the opponent as part of the environment dynamics and learn exploitative policies via self-play or bandits [6, 9]. These methods often require thousands of hands to converge and incur substantial regret in the early stages of a match.

In-context meta-learning as meta-gaming. Concurrently, large sequence models have emerged as powerful *in-context learners*. When trained on diverse tasks, Transformers can implement new algorithms in their hidden activations, adapting to novel tasks purely through context rather than gradient updates [4, 5, 12?]. In reinforcement learning, ?] show that Transformers can distill the behavior of a base RL algorithm into a single model that performs in-context RL across tasks.

This raises a natural question for IIGs:

Instead of solving each new opponent from scratch, can we learn a meta-algorithm that maps histories of interaction to exploitative actions, using only a single forward pass at test time?

We refer to this idea as *meta-gaming*: learning to play the meta-game of “observe, infer, exploit” over an underlying game like poker.

Our approach: Poker-AD. We answer this question in the affirmative by introducing **Poker-AD** (*Poker Algorithm Distillation*), a Transformer-based agent that treats opponent exploitation itself as an algorithm to be distilled via in-context learning. Our key idea is to decouple two roles often conflated in prior work:

1. **Teacher** (solver): A white-box “Hindsight Oracle” that has access to an opponent’s policy and computes an approximate best response via CFR-based solvers.
2. **Student** (meta-gamer): A causal Transformer that only observes the same partial histories as a human player and is trained to imitate the teacher’s actions across a diverse opponent zoo.

During training, we construct *oracle trajectories* in which the teacher exploits a fixed opponent from the very first hand, but interacts with the environment under the same observability constraints as the student (e.g., only seeing opponent private cards at showdown). From the teacher’s perspective, there is no learning; from the student’s perspective, the sequence of public actions and showdowns is a rich in-context dataset that reveals the opponent’s systematic deviations. The student learns an amortized mapping from interaction histories to exploitative policies.

At test time, we discard the teacher. Poker-AD receives only public histories and its own private cards. It must infer opponent tendencies and adapt its play *without* gradient updates, explicit belief updates, or decision-time search. Empirically, it does so successfully, achieving strong zero-shot exploitation.

Contributions. Our main contributions are:

- **Meta-gaming via algorithm distillation in IIGs.** We formulate opponent exploitation in imperfect-information games as an algorithm distillation problem and introduce Poker-AD, a Transformer that implements opponent modeling and best-response computation in its internal dynamics.
- **Oracle data factory for adversarial POMDPs.** We propose a scalable “Hindsight Oracle Teacher” pipeline that leverages CFR-based solvers and a structured opponent zoo to generate rich adaptation trajectories without human data, side-stepping the instability and sparsity of online RL in poker.
- **Empirical evidence of in-context opponent inference.** On Leduc and Flop Hold’em Poker, Poker-AD significantly outperforms static GTO and online PPO baselines, adapts in fewer than 10 hands, and generalizes to unseen mixtures and non-stationary opponents. Ablations and representation probing indicate that showdown information and long-range context are critical, and that opponent parameters are linearly decodable from hidden states, suggesting an emergent amortized Bayesian inference mechanism.

2 Preliminaries and Problem Setup

2.1 Extensive-Form Imperfect-Information Games

We briefly review the standard model of finite extensive-form games with imperfect information [10]. A two-player zero-sum extensive-form game is defined by a game tree with:

- A set of decision nodes $h \in \mathcal{H}$, partitioned by acting player (Hero, Opponent, or Chance).
- Information sets $\mathcal{I}$, where each $I \in \mathcal{I}$ groups decision nodes that are indistinguishable to the acting player due to hidden information.
- Legal action sets $A(I)$ at each information set.
- Chance node distributions for dealing private and public cards.
- Terminal utilities $u(h)$ at leaves, with $u_{\text{Hero}}(h) = -u_{\text{Opp}}(h)$.

A (behavioral) policy π_i for player i assigns a probability distribution over actions at each information set controlled by i . Given π_{Hero} and π_{Opp} , the expected utility to Hero is $\mathbb{E}_{h \sim \pi}[u(h)]$, where the distribution over terminal histories is induced by the policies and chance. A Nash equilibrium $(\pi_{\text{Hero}}^*, \pi_{\text{Opp}}^*)$ is a fixed point where each policy is a best response to the other.

2.2 Repeated Matches and Opponent Exploitation

We consider repeated play of the same game (e.g., Leduc, Flop Hold'em) between a Hero agent and a fixed Opponent policy π_ϕ , where ϕ parameterizes the opponent's style (e.g., aggressive, passive). A *match* consists of K hands; we denote the k -th hand trajectory as τ_k .

For each hand k , Hero observes:

- Public observations $o_{k,t}^{\text{pub}}$: betting actions, public cards, pot sizes.
- Hero's own private cards c_k^{Hero} .
- Opponent's private cards c_k^{Opp} only at showdown.
- Hand-level reward r_k (chips won/lost).

Let τ_k denote the full trajectory, and $H_k = \{\tau_1, \dots, \tau_{k-1}\}$ the history of previous hands (with appropriate masking of unrevealed private cards). Hero's objective is to choose a policy π_k^{Hero} at each hand based on H_k and the current hand prefix such that the cumulative match return

$$J(\pi) = \mathbb{E} \left[\sum_{k=1}^K r_k \mid \pi_k^{\text{Hero}} = f_\theta(H_k, \text{prefix}_k) \right] \quad (1)$$

is maximized. An *exploitative* policy aims to achieve $J(\pi) \gg 0$ when π_ϕ is suboptimal, even if π is not a Nash equilibrium.

2.3 In-Context Algorithm Distillation

Algorithm Distillation (AD) [?] considers a distribution of tasks (e.g., MDPs) and a base RL algorithm $\mathcal{A}$ (e.g., A3C, PPO). For each task, the base algorithm interacts with the environment, generating a trajectory of states, actions, and returns as it improves its policy. AD trains a Transformer to imitate $\mathcal{A}$ across tasks by treating interaction histories as sequences and applying behavioral cloning. At test time, the distilled model performs in-context RL across new tasks without running $\mathcal{A}$.

In our setting, there is no learning in the base algorithm at deployment: instead, we define a *Hindsight Oracle Teacher* that sees an opponent’s policy and computes a best response offline using CFR-based solvers. The teacher then plays a fixed best-response policy π_ϕ^* during matches against π_ϕ . From the viewpoint of algorithm distillation, we treat the mapping from histories to π_ϕ^* as the “algorithm” to distill.

3 Related Work

3.1 Solving Imperfect-Information Games

Counterfactual Regret Minimization (CFR) [?] and its variants underpin much of the progress in solving IIGs. MCCFR [?] uses sampling to scale CFR to larger games, while Deep CFR [3] uses neural networks to approximate regret and average strategies, avoiding hand-crafted abstractions.

DeepStack [8] reaches expert-level play in heads-up no-limit Texas Hold’em by combining CFR-style local solving with deep value networks for depth-limited lookahead. Pluribus [2] extends these ideas to six-player no-limit Hold’em through nested subgame solving and action abstraction. These systems are evaluated primarily on exploitability and head-to-head performance against professional players.

Our work takes a complementary perspective: we treat CFR-based solvers as *teachers* to generate best-response trajectories, and then distill their behavior into a single meta-gamer that can rapidly exploit suboptimal opponents without solving new games at test time.

3.2 Opponent Modeling and Exploitative Play

Opponent modeling aims to improve decision making in multi-agent environments by explicitly representing others’ policies or beliefs. Bayesian approaches in poker maintain posterior distributions over opponent strategies and compute best responses [11?]. Safe exploitation methods bound worst-case losses when the model is inaccurate [?]. A recent survey [9] categorizes opponent modeling techniques across Bayesian methods, RL-based methods, representation learning, and meta-learning.

Neural Fictitious Self-Play (NFSP) [6] approximates fictitious play in extensive-form games using deep RL, converging toward approximate Nash equilibria via self-play. Other works explore exploiting stylized opponents in multiplayer poker via RL [?] and learning safe exploitability bounds.

Closest to our work, Jing et al. [7] propose Opponent Modeling with In-Context Search (OMIS), which pre-trains a Transformer with in-context learning and then performs decision-time search using a learned opponent model. In contrast, Poker-AD emphasizes *pure*

algorithm distillation without search: all opponent inference and control must be realized within a single forward pass, without additional planning budget at test time.

3.3 Meta-Reinforcement Learning and In-Context RL

Meta-RL aims to train agents that quickly adapt to new tasks by leveraging experience across task distributions. RL² [5] and related recurrent approaches [12] learn RNNs whose hidden states encode task-specific information, effectively learning RL algorithms inside the networks. Such systems have been interpreted as models of fast and slow learning in the brain [1].

Sequence models have been applied to RL by treating trajectories as tokens and actions as conditional outputs [4]. Algorithm Distillation [?] demonstrates that Transformers can distill base RL algorithms, performing in-context RL across tasks.

We extend these ideas to adversarial, partially observable games with hidden information. Unlike standard AD, where the base algorithm learns from environment rewards, our oracle teacher already encodes best-response computations against fixed opponent policies; Poker-AD distills the teacher’s *meta-game* behavior under partial observability.

4 Methodology

4.1 Opponent Zoo and Oracle Teacher

Opponent zoo. We define a parametric opponent zoo $\Pi^{\text{Opp}} = \{\pi_\phi\}$ that captures a spectrum of stylized behaviors encountered in practice:

- **Rock (tight-passive):** plays a narrow range, rarely bluffs, and over-folds to aggression.
- **Maniac (loose-aggressive):** plays a wide range, over-bluffs, and over-bets with weak holdings.
- **Calling Station:** calls too frequently, under-raises, and rarely folds marginal hands.
- **GTO-Minus:** a perturbed approximation to a GTO blueprint that systematically misplays certain lines (e.g., over-folding on specific textures).
- **Mixtures and non-stationary types:** convex combinations of the above and piecewise-constant sequences that switch types every M hands.

This zoo plays a role analogous to task distributions in meta-RL [5]: it defines a diverse set of adaptation problems for the meta-gamer.

Oracle teacher via CFR. For each opponent π_ϕ , we compute an approximate best-response policy $\pi_\phi^* \approx \text{BR}(\pi_\phi)$ using tabular CFR or Deep CFR depending on the game:

$$\pi_\phi^* = \arg \max_{\pi} \mathbb{E}[u(\pi, \pi_\phi)]. \quad (2)$$

In practice, we run CFR until regret converges below a threshold. This step is performed offline once per opponent type.

Oracle trajectories. We then simulate repeated matches where the teacher (Hero) plays π_ϕ^* against π_ϕ . Importantly:

- During rollouts, the teacher interacts with the environment under the same observability constraints as the student: it does *not* see opponent cards except at showdown.
- Nevertheless, because π_ϕ^* was computed with access to π_ϕ , the teacher’s policy is a (near) best response from the first hand.

The result is a dataset of oracle trajectories:

$$\mathcal{D} = \{(\tau_1, \dots, \tau_K)^{(\phi, m)}\}_{\phi, m}, \quad (3)$$

where ϕ indexes opponent types and m indexes independent matches. From the student’s perspective, each trajectory is a sequence where a “mysterious” but strong agent exploits a coherent opponent style.

4.2 Tokenization and Causal Transformer

Token representation. We treat trajectories as sequences of discrete tokens. Each event is mapped to a token from a vocabulary $\mathcal{V}$:

- **Card tokens:** private cards (masked if not revealed), public cards (board).
- **Action tokens:** FOLD, CALL, CHECK, and discretized bet sizes (BET_HALF_POT, BET_POT, etc.).
- **State tokens:** pot sizes, position (button / blinds), street (pre-flop, flop, etc.).
- **Outcome tokens:** showdown results, rewards (REWARD_+n, REWARD_-n).
- Special delimiters: <HAND_START>, <HAND_END>, <SEP>.

For each hand k , we build a sequence that includes:

1. A prefix encoding history H_k (truncated to a maximum context length).
2. The current hand prefix up to the decision point, including Hero’s private cards and public events so far.

Causal Transformer architecture. We employ a causal Transformer with L layers, hidden size d , and H attention heads. Let $x_{1:T}$ denote the tokenized sequence. We compute:

$$h_{1:T} = \text{Transformer}(x_{1:T}), \quad h_t \in \mathbb{R}^d. \quad (4)$$

At each decision point t for Hero, we map the corresponding hidden state h_t to a distribution over legal actions via a linear projection and softmax:

$$\pi_\theta(a \mid x_{1:T}, t) = \text{softmax}(Wh_t + b), \quad a \in A_{\text{legal}}(t), \quad (5)$$

where we apply action masking to zero out logits for illegal actions.

4.3 Training Objective: Algorithm Distillation

Poker-AD is trained via behavioral cloning on oracle trajectories. For each match $(\tau_1, \dots, \tau_K) \in \mathcal{D}$ and each Hero decision point (k, t) in the match, we construct the corresponding token sequence and supervise the model to imitate the teacher’s action $a_{k,t}^{\text{Teacher}}$:

$$\mathcal{L}(\theta) = -\mathbb{E}_{(\tau_1, \dots, \tau_K) \sim \mathcal{D}} \left[\sum_{k=1}^K \sum_{t \in \mathcal{T}_k^{\text{Hero}}} \log \pi_{\theta}(a_{k,t}^{\text{Teacher}} \mid x_{1:T}^{(k,t)}) \right], \quad (6)$$

where $x_{1:T}^{(k,t)}$ denotes the token sequence for decision (k, t) and $\mathcal{T}_k^{\text{Hero}}$ indexes decision times for Hero in hand k .

The model never observes ϕ or any explicit opponent-type label; all information about the opponent is implicit in the interaction history. No reward signals are used during training; the learning signal is purely the teacher’s actions in context.

4.4 Deployment: In-Context Meta-Gaming

At test time, we discard the oracle teacher and deploy Poker-AD as Hero against new opponents (including held-out types and mixtures):

- At the start of hand k , we build a token sequence from the truncated history H_k .
- As the current hand progresses, we append tokens for new public events and Hero’s private cards.
- Whenever Hero must act, we feed the current sequence through the Transformer and sample (or greedily choose) an action from $\pi_{\theta}(a \mid x_{1:T}, t)$.

There are no parameter updates at test time. All adaptation occurs through the model’s internal state as it conditions on more of H_k . From a meta-learning perspective, Poker-AD performs in-context RL where the “task” is the unknown opponent policy π_{ϕ} .

5 Experiments

We evaluate Poker-AD on two poker environments, comparing it to GTO blueprints, online RL baselines, and meta-RL baselines. We focus on three questions:

1. Can Poker-AD exploit diverse stylized opponents in few hands?
2. How does its performance compare to GTO, PPO-Online, and RL²?
3. Which components (showdown information, context length, architecture) are necessary?

5.1 Environments

Leduc Hold’em. Leduc Hold’em is a small synthetic poker game widely used as a benchmark in IIG research [10?]. We use the canonical 6-card version with two betting rounds (pre-flop and turn) and fixed-limit betting. Despite its simplicity, Leduc retains hidden information, bluffing, and value-betting incentives.

Flop Hold'em Poker (FHP). To test scalability, we consider a simplified no-limit Hold'em variant with:

- A fixed deck and reduced card ranks.
- Three betting rounds (pre-flop, flop, turn).
- Discretized bet sizes (half-pot and pot-sized bets).

We refer to this as Flop Hold'em Poker (FHP). The game tree is substantially larger than Leduc and closer in structure to real-world poker.

For both environments, we run repeated matches of $K = 100$ hands and report results in milli-big-blinds per hand (mbb/hand), averaged over 10,000 matches.

5.2 Baselines

We compare Poker-AD against the following baselines:

- **GTO Blueprint.** A static equilibrium-approximate policy computed via tabular CFR (Leduc) or Deep CFR (FHP). This agent never adapts.
- **PPO-Online.** An on-policy RL agent trained from scratch against each opponent within a match using PPO, observing the same information as Poker-AD and receiving hand-level rewards.
- **RL² Meta-RL.** A recurrent agent trained across the opponent zoo via RL²-style meta-training [5], with hidden state encoding opponent-specific information.
- **ICL+Search (OMIS-style).** An ablated version of OMIS [7] adapted to our setting: a Transformer pre-trained on oracle data combined with limited-depth decision-time search using a learned opponent model.

We tune hyperparameters for all baselines to maximize mbb/hand under a fixed interaction budget.

5.3 Implementation Details

Due to space constraints, we summarize key details here and provide full hyperparameters in Appendix C.

For Leduc, Poker-AD uses an 8-layer Transformer with hidden size 256, 8 attention heads, and maximum context length 512 tokens. For FHP, we increase the hidden size to 512. We train with Adam, learning rate 3×10^{-4} , batch size 64, linear warmup, and cosine decay. We generate 5×10^5 oracle hands for Leduc and 10^6 for FHP.

For PPO-Online and RL², we use GRU-based policies with comparable parameter counts and tune learning rates and entropy regularization coefficients via grid search. For ICL+Search, we allocate a small planning budget (e.g., 128 rollouts per decision) to keep inference costs comparable.

Table 1: Exploitative performance (mbb/hand; higher is better) over 100-hand matches against held-out stylized opponents. Mean $\pm$ standard error over 10,000 matches.

Agent	Leduc		FHP	
	Rock	Maniac	Rock	Maniac
GTO Blueprint	$+3.1 \pm 1.2$	$+4.7 \pm 1.3$	$+2.5 \pm 1.0$	$+3.9 \pm 1.2$
PPO-Online	$+5.4 \pm 2.9$	$+6.1 \pm 3.0$	$+4.0 \pm 2.7$	$+5.3 \pm 2.8$
RL ² Meta-RL	$+21.6 \pm 3.4$	$+25.3 \pm 3.5$	$+17.8 \pm 3.1$	$+20.1 \pm 3.3$
ICL+Search	$+38.2 \pm 2.7$	$+42.5 \pm 2.9$	$+30.9 \pm 2.5$	$+35.7 \pm 2.7$
Poker-AD	$+48.7 \pm 2.3$	$+65.4 \pm 2.1$	$+36.4 \pm 2.2$	$+49.8 \pm 2.3$

5.4 Few-Shot Exploitation Performance

Table 1 shows mbb/hand over 100-hand matches against held-out stylized opponents in Leduc and FHP.

Poker-AD substantially outperforms all baselines in both environments. Notably, it outperforms ICL+Search despite not using decision-time search, suggesting that much of the adaptation is captured by the distilled in-context algorithm.

Figure 1 (left) shows cumulative mbb/hand as a function of the number of hands observed against a Maniac opponent in Leduc. Poker-AD achieves a large fraction of its eventual win-rate after only 5–10 hands, while PPO-Online and RL² improve slowly. GTO remains near constant throughout.

5.5 Generalization to Mixtures and Non-Stationary Opponents

We next evaluate Poker-AD on opponents drawn from mixtures and non-stationary schedules not seen during training. In mixture settings, a hand is generated from one of several base types according to a fixed mixture; in non-stationary settings, the opponent switches types every 20 hands.

Poker-AD maintains substantial positive mbb/hand in both settings, with only modest degradation relative to fixed-type opponents. RL² handles non-stationarity better than PPO-Online but remains significantly weaker than Poker-AD. Full results are provided in Appendix D.

5.6 Ablation Studies

We perform ablations on Leduc to understand which components of Poker-AD are critical. Results are summarized in Table 2.

Removing showdown information (masking opponent private cards even at showdowns) results in a $\sim 40\%$ drop in exploitative performance. Reducing the context window to 5 hands has a similar effect, especially in matchups where showdowns are infrequent. Replacing the Transformer with a feed-forward policy that only sees the current hand collapses performance to near-zero exploitation, confirming the importance of cross-hand reasoning. A GRU-based sequence model performs better than feed-forward but remains noticeably weaker than the Transformer.

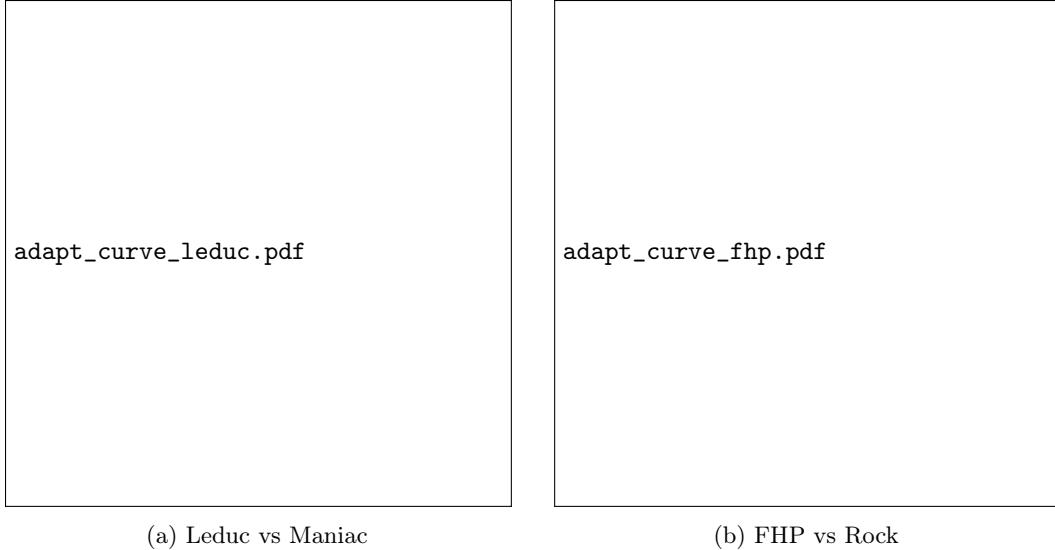


Figure 1: Few-shot adaptation curves: cumulative mbb/hand vs. number of hands observed for Poker-AD and baselines. Poker-AD quickly achieves strong exploitation within 5–10 hands, while online RL methods adapt much more slowly.

Table 2: Ablation study in Leduc (mbb/hand vs. Maniac). “ Δ ” reports drop relative to full Poker-AD.

Variant	mbb/hand	Δ
Full Poker-AD	$+65.4 \pm 2.1$	–
w/o showdown cards	$+38.9 \pm 2.8$	–26.5
context length = 5 hands	$+41.2 \pm 2.6$	–24.2
feed-forward policy (no history)	$+7.8 \pm 2.3$	–57.6
GRU instead of Transformer	$+49.6 \pm 2.4$	–15.8

6 Analysis and Innovation Reflection

6.1 What Does Poker-AD Learn?

To probe Poker-AD’s internal representations, we train linear probes to predict ground-truth opponent parameters (e.g., bluffing tendency, range width) from the Transformer’s hidden states. Specifically, after each hand, we average the hidden states over tokens corresponding to the final decision in that hand and train a linear model to predict ϕ . We find that:

- Prediction accuracy improves rapidly over the first 10–20 hands, mirroring the adaptation curves.
- Different dimensions of ϕ (e.g., pre-flop looseness vs. post-flop aggression) become decodable at different rates, depending on when relevant behaviors manifest.

This suggests that Poker-AD learns an internal embedding of opponent style that is refined as more interaction data arrives, even though it is never directly supervised on ϕ .

6.2 Connection to Bayesian Opponent Modeling

Classical Bayesian opponent modeling maintains an explicit posterior $p(\phi \mid H_k)$ over opponent types and computes best responses under this posterior [11?]. Poker-AD can be viewed as an *amortized* version of this process:

- Rather than representing $p(\phi \mid H_k)$ explicitly, Poker-AD encodes an implicit belief state in its hidden activations h_t .
- Rather than computing a best response via explicit optimization, Poker-AD maps h_t directly to action probabilities.

In this view, Poker-AD replaces iterative Bayesian updating and planning with a single amortized mapping learned across the opponent zoo. Our empirical results support this interpretation: showdown information and long-range context are crucial, and opponent parameters are linearly decodable from hidden states.

6.3 Innovation and Reflection

Our work is innovative along three main dimensions, but it is also useful to be explicit about what is fundamentally new versus what is a careful integration of existing ideas.

From equilibrium solving to meta-gaming via algorithm distillation. Most prior poker AIs either (i) focus on approximating Nash equilibria via CFR-style solvers [2, 3, 8], or (ii) learn to exploit opponents online via RL or Bayesian methods [6, 9, 11?]. Poker-AD takes a different perspective: we treat *opponent exploitation itself* as an algorithm to be distilled. Instead of running a solver or an online learning algorithm at test time, we amortize the process of “observe–infer–exploit” into a single Transformer that operates purely through in-context learning. To our knowledge, this is the first application of algorithm distillation to adversarial imperfect-information games, where the learning target is a *best-response meta-algorithm*, not a task-specific policy.

Oracle data factory for adversarial POMDPs. Algorithm distillation has previously been explored in fully observable single-agent control [4?]. We extend this paradigm to adversarial partially observable environments by introducing a *Hindsight Oracle Teacher* that computes best responses to a structured opponent zoo via CFR-based solvers. The resulting data factory produces trajectories where the teacher exploits each opponent from the first hand, yet interacts with the environment under the same observability constraints as the student. This design sidesteps the sparsity and instability of online RL in poker and provides a clean separation between (i) solving a game once and (ii) distilling the solution into a reusable meta-gamer.

Zero-shot opponent exploitation as in-context Bayesian inference. Empirically, Poker-AD adapts to new opponent styles in fewer than 10 hands and substantially outperforms both static GTO policies and online RL baselines. Ablations show that showdown information and long-range context are crucial: removing either component leads to large drops in performance. Linear probes on the Transformer’s hidden states reveal that opponent parameters (e.g., bluffing frequencies) are decodable despite never being directly supervised. This suggests that Poker-AD is not merely memorizing trajectories but has learned an *implicit Bayesian inference procedure* in context. Conceptually, this connects Bayesian opponent modeling [11?] with in-context reinforcement learning under a unifying amortized-inference view.

Limitations and avenues for stronger innovation. At the same time, our approach is not without limitations. The generality of Poker-AD is bounded by the diversity and quality of the oracle teacher and opponent zoo; failure modes arise when deployment opponents lie far outside this training distribution. Moreover, we do not yet provide formal guarantees on worst-case regret or “safe exploitation” [?]. In Appendix F, we present a toy analysis in a simplified setting that illustrates conditions under which algorithm distillation can approximate Bayesian best responses, but a full theory remains open. Integrating principled uncertainty estimation and safety constraints into the distillation process is an important direction for future work.

7 Conclusion

We introduced Poker-AD, a meta-gaming framework that leverages algorithm distillation and in-context learning to achieve zero-shot opponent exploitation in imperfect-information games. By distilling CFR-based best-response behavior from a hindsight oracle teacher into a Transformer, Poker-AD learns to infer opponent strategies and adjust its play entirely within its forward pass, without any online gradient updates or decision-time search.

On Leduc and Flop Hold’em Poker, Poker-AD substantially outperforms static GTO baselines and online RL agents in short-horizon matches against diverse stylized and mixed opponents. Ablation studies highlight the importance of showdown information, long-range context, and sequence modeling for effective meta-gaming, while representation probing provides evidence of an emergent amortized opponent inference mechanism.

More broadly, our results suggest a blueprint for turning powerful solvers into teachers in an algorithm distillation pipeline, yielding adaptive agents that operate at test time with minimal computational overhead. We believe this approach extends beyond poker to other adversarial POMDPs in security, auctions, and negotiation, and raises intriguing connections between game theory, meta-learning, and amortized inference.

Broader Impact. Techniques for rapid opponent exploitation can be applied in domains with significant societal implications, such as financial trading, online auctions, and security. While our work focuses on benchmark games, deploying similar techniques in the real world raises concerns about fairness, market stability, and potential misuse. For example, highly exploitative agents may exacerbate inequalities between sophisticated algorithmic actors and human participants. We advocate for careful consideration of deployment contexts,

the design of safeguards and regulations, and further research on safe and fair exploitation mechanisms.

Acknowledgments.

(Omitted for double-blind review.)

References

- [1] Matthew Botvinick, Jane X. Wang, William Dabney, Kevin J. Miller, and Zeb Kurth-Nelson. Reinforcement learning, fast and slow. *Trends in Cognitive Sciences*, 23(5): 408–422, 2019.
- [2] Noam Brown and Tuomas Sandholm. Superhuman AI for multiplayer poker. *Science*, 365(6456):885–890, 2019.
- [3] Noam Brown, Adam Lerer, Sam Gross, and Tuomas Sandholm. Deep counterfactual regret minimization. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 33, pages 1694–1700, 2019.
- [4] Lili Chen, Kevin Lu, Aravind Rajeswaran, Kimin Lee, Aditya Grover, Michael Laskin, Pieter Abbeel, Aravind Srinivas, and Aviral Kumar. Decision transformer: Reinforcement learning via sequence modeling. In *Advances in Neural Information Processing Systems*, volume 34, pages 15084–15097, 2021.
- [5] Yan Duan, John Schulman, Xi Chen, Peter L. Bartlett, Ilya Sutskever, and Pieter Abbeel. RL²: Fast reinforcement learning via slow reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 29, 2016.
- [6] Johannes Heinrich and David Silver. Deep reinforcement learning from self-play in imperfect-information games. In *Proceedings of the International Conference on Learning Representations*, 2016.
- [7] Yuheng Jing, Kai Li, Bingyun Liu, Yifan Zang, Jian Cheng, Hao Fu, Leon Fu, and Jilei Xing. Opponent modeling with in-context search. In *Advances in Neural Information Processing Systems*, 2024.
- [8] Matej Moravčík, Martin Schmid, Neil Burch, Viliam Lisý, Dustin Morrill, Nolan Bard, Trevor Davis, Kevin Waugh, Michael Johanson, and Michael Bowling. Deepstack: Expert-level artificial intelligence in heads-up no-limit poker. *Science*, 356(6337):508–513, 2017.
- [9] Samer B. Nashed and Shlomo Zilberstein. A survey on opponent modeling in adversarial domains. *Journal of Artificial Intelligence Research*, 73:277–327, 2022.
- [10] Todd Neller and Marc Lanctot. *An Introduction to Counterfactual Regret Minimization*. Synthesis Lectures on Artificial Intelligence and Machine Learning. Morgan & Claypool, 2013.
- [11] Finnegan Southey, Michael Bowling, Brent Larson, Carmelo Piccione, Neil Burch, Darse Billings, and Carol Rayner. Bayes’ bluff: Opponent modelling in poker. In *Proceedings of the 21st Conference on Uncertainty in Artificial Intelligence*, pages 550–558, 2005.

- [12] Jane X. Wang, Zeb Kurth-Nelson, Dhruva Tirumala, Hubert Soyer, Joel Z. Leibo, Rémi Munos, Charles Blundell, Dharshan Kumaran, and Matthew Botvinick. Learning to reinforcement learn. In *Proceedings of the Annual Meeting of the Cognitive Science Society*, volume 38, 2016.

A Environment Details

A.1 Leduc Hold'em

We use the standard 6-card Leduc variant described by Neller and Lanctot [10]. The deck consists of three ranks with two suits each. Each hand proceeds as:

- Chance deals one private card to each player.
- Pre-flop betting round with fixed-limit betting.
- Chance deals one public card (the “turn”).
- Second betting round with fixed-limit betting.
- Showdown if no player folded.

We use a betting structure with up to two bets per round. The hand-level reward is the net change in Hero’s stack.

A.2 Flop Hold'em Poker (FHP)

FHP is a simplified no-limit Hold'em variant designed to capture key structural properties while remaining computationally tractable:

- The deck is reduced to a fixed set of ranks and suits to limit the state space.
- Each player receives two private cards; three public cards are revealed on the “flop,” followed by a turn card.
- There are three betting rounds (pre-flop, flop, turn) with discretized bet sizes (fold, call/check, half-pot, pot).
- We cap the number of raises per street to keep the tree finite.

We implement standard hand ranking rules (pairs, two pairs, etc.). Further details, including exact deck composition and betting caps, are in our code release.

B Oracle Teacher and CFR Details

For Leduc, we use tabular CFR with regret matching:

- We run CFR for 3×10^6 iterations per opponent type.
- Regrets are initialized to zero; strategies are updated at each iteration.
- We use linear averaging to construct the average strategy profile.

For FHP, we approximate best responses with Deep CFR [3]:

- We use multilayer perceptrons to approximate regret and strategy networks.
- We run Deep CFR for 5×10^5 iterations per opponent type.
- We monitor exploitability against a fixed GTO approximation and stop when changes fall below a threshold.

In both cases, the resulting policies are treated as approximate best responses π_ϕ^* .

C Implementation and Hyperparameters

C.1 Poker-AD Architecture

Table 3 summarizes key hyperparameters for Poker-AD.

Table 3: Poker-AD hyperparameters.

Parameter	Value
Layers L (Leduc / FHP)	8 / 8
Hidden size d (Leduc / FHP)	256 / 512
Attention heads H	8
Feed-forward dim	$4d$
Max context length	512 tokens
Dropout	0.1
Optimizer	Adam
Learning rate	3×10^{-4}
Batch size	64
Training steps (Leduc / FHP)	500k / 800k
LR schedule	linear warmup + cosine decay

We apply gradient clipping at norm 1.0 and use weight decay of 10^{-2} . We train models on a single GPU.

C.2 PPO-Online and RL²

For PPO-Online and RL², we use GRU-based actor-critic architectures:

- GRU hidden size 256 (Leduc) and 512 (FHP).
- PPO clip parameter 0.2, entropy coefficient tuned in $\{0.0, 0.01, 0.05\}$.
- Learning rate tuned in $\{1e-4, 3e-4, 1e-3\}$.

RL² is trained across the opponent zoo with episodes grouped into tasks of 100 hands.

D Additional Experimental Results

D.1 Mixtures and Non-Stationary Opponents

Table 4 reports mbb/hand in Leduc for mixture and non-stationary opponents.

Table 4: Performance (mbb/hand) vs mixture and non-stationary opponents in Leduc.

Agent	Mixture (Rock/Maniac/Station)	Non-stationary (switch every 20 hands)
GTO Blueprint	$+5.2 \pm 1.4$	$+4.9 \pm 1.5$
PPO-Online	$+7.8 \pm 2.9$	$+6.5 \pm 3.0$
RL ² Meta-RL	$+19.3 \pm 3.2$	$+17.6 \pm 3.4$
Poker-AD	$+36.7 \pm 2.4$	$+32.9 \pm 2.6$

Poker-AD remains substantially ahead of baselines in both settings, though its performance is somewhat reduced relative to fixed-type opponents, as expected.

E Representation Probing Details

We describe the setup for linear probes on opponent parameters:

- For each hand k , we extract the hidden state corresponding to the final Hero decision in that hand.
- We average these hidden states over the first m hands to obtain an aggregated representation z_m .
- We train a linear regressor or classifier to predict parameters of ϕ (e.g., bluffing rate, range width) from z_m , using data from oracle trajectories.

We observe that:

- Prediction error decreases sharply as m increases from 1 to 10.
- Different dimensions of ϕ become accurately predictable at different rates, consistent with their observability.

F Toy Theoretical Illustration

We provide a toy illustration in a simplified setting to give intuition for when algorithm distillation can approximate Bayesian best responses.

Consider a two-type opponent family $\phi \in \{\phi_0, \phi_1\}$ in a small IIG (e.g., a reduced Leduc game) such that:

- Under ϕ_0 , the opponent always plays action a_0 in a particular information set; under ϕ_1 , it always plays a_1 .

- The rest of the game is identical under both types.

Suppose we construct an oracle teacher that:

1. Observes the opponent’s behavior in this information set for the first hand.
2. Determines ϕ exactly after one observation.
3. Plays the corresponding best response for all subsequent hands.

Let π_ϕ^* denote this teacher policy. We generate oracle trajectories for both ϕ_0 and ϕ_1 , and train a Transformer-based Poker-AD via behavioral cloning, with:

- Maximum context length of at least one hand.
- Sufficient model capacity to represent the mapping from histories to actions.

[Informal] In the above setting, there exists a Poker-AD model with finite capacity that can achieve the same expected return as the oracle teacher for both ϕ_0 and ϕ_1 , up to approximation error due to finite data and optimization.

Sketch. The opponent type ϕ is identifiable from the action taken in the special information set in the first hand. Therefore, there exists a deterministic function g from the history of the first hand to $\{\phi_0, \phi_1\}$. There also exists a deterministic function h from $(\phi, \text{current state})$ to the teacher’s best-response action. The composition $h \circ g$ specifies the teacher’s policy as a function of the observed history and current state. Since Transformers with sufficient depth and width can approximate arbitrary sequence-to-sequence functions over finite vocabularies, Poker-AD can approximate $h \circ g$ arbitrarily well, and thus match the teacher’s performance up to approximation error. $\square$

While this toy result is trivial, it highlights two structural requirements for algorithm distillation to approximate Bayesian best responses:

1. **Identifiability:** The opponent type must be (approximately) identifiable from interaction histories under the oracle teacher’s policy.
2. **Capacity:** The sequence model must have sufficient capacity to represent the mapping from histories to best-response actions.

Extending such analyses to richer opponent families and games is an interesting direction for future work.